CS-GY 6643 Project 3: Computer Vision

Total Marks: 100 Submission Deadline: November 8th

Marks Distribution:

This project consists of **4 questions**: Question 1 is **10 points**, Question 2 is **10 points**, Question 3 is **15 points**, and the Kaggle competition (Question 4) is **65 points**, for a total of **100 points**.

Question	Topic	Points
1	CNN Feature Visualization & Interpretation	10
2	Multi-Car Speed Estimation with Detection & Optical Flow	10
3	Person & Dog Tracking with Object Detection & Motion Analysis	15
4	Penalty Kick Prediction Kaggle Competition	65
	Total	**100**

Submission Instructions

Please read the following instructions carefully to ensure your project is submitted correctly.

- **Single Notebook Format:** Your entire project must be consolidated into one single Jupyter or Google Colab notebook (`.ipynb` file), including all code, visual outputs, and written explanations.
- **Submission Process (File AND Link):** On Brightspace, you must submit both your final `.ipynb` file (as a file upload) and a shareable link to it (in the text box). Ensure that link sharing is set to "Anyone with the link can view."
- **Report Format:** Your notebook serves as the final report. Use Markdown cells to explain your methods, results, and challenges clearly.
- **LLM Usage Policy:** If you use an LLM (ChatGPT, Claude, Gemini, etc.) for assistance, include a shareable link or PDF of the chat history along with your submission.

Project 3: Computer Vision

Deadline: November 5th, 2026

Question 1: CNN Feature Visualization & Interpretation (10 pts)

Question 1 Resources

We have seen that convolutional networks produce increasingly abstract feature maps as we move deeper into the network. In this question, you will **visualize** and **compare** these internal activations for three different architectures and for two different images.

Reference images and example (given)

Figure 1 shows two input images you should use throughout this question: (a) a **tiger** image and (b) a **human face** image. Figure 2 shows an **example** visualization already done for you using **VGG16** on the tiger image at three depths (early, mid, deep).

Note that activations from the **last convolutional layer** are often highly abstract and may appear very sparse or uninterpretable when visualized directly. To better understand which image regions drive the network's prediction, you should use **Grad-CAM** (Gradient-weighted Class Activation Mapping), which highlights the most class-relevant spatial locations.



(a) Input image 1: Tiger

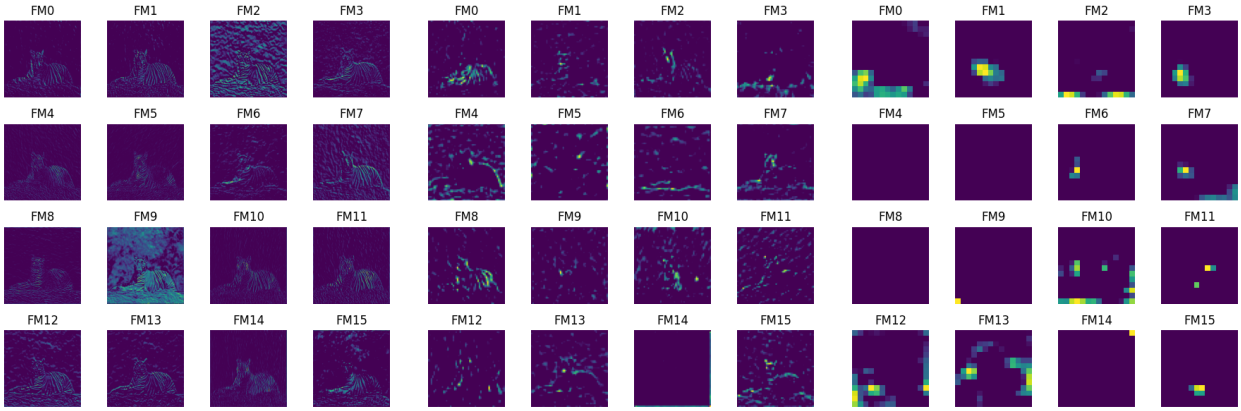


(b) Input image 2: Face

Figure 1: Source images to use for visualization.

Grad-CAM Visualization

Gradient-weighted Class Activation Mapping (Grad-CAM) highlights image regions that most influence the model's prediction by weighting each channel of the final convolutional layer using the gradient of the target class score. This produces an interpretable heatmap showing where the network "looks" when classifying an image.



(a) VGG16, tiger, early layer (`features[3]`) (b) VGG16, tiger, mid layer (`features[15]`) (c) VGG16, tiger, deep layer (`features[28]`)

Figure 2: Example feature maps for VGG16 on the tiger image at different depths.

Grad-CAM Implementation Code Example (VGG16)

```
import torch, cv2, numpy as np, matplotlib.pyplot as plt

model.eval()
target_layer = model.features[28] # last conv layer
fmap, grads = {}, {}

def fwd_hook(m, i, o): fmap["value"] = o.detach()
def bwd_hook(m, gin, gout): grads["value"] = gout[0].detach()

# register hooks
h1 = target_layer.register_forward_hook(fwd_hook)
h2 = target_layer.register_backward_hook(bwd_hook)

# forward & backward
out = model(input_tensor)
pred_class = out.argmax(dim=1).item()
model.zero_grad()
out[0, pred_class].backward()

# feature maps (A) & gradients (G)
A, G = fmap["value"], grads["value"]
alpha = G.mean(dim=(2,3)).squeeze(0) # channel importance
topk = alpha.topk(15).indices # pick top 15 channels

img_np = np.array(img).astype(np.float32) / 255.0
H, W = img.size[1], img.size[0]

fig, axes = plt.subplots(3,5,figsize=(14,8))
for idx, ax in enumerate(axes.flat):
    ch = topk[idx].item()
    cam = torch.relu(alpha[ch] * A[0,ch,:,:]).cpu().numpy()
    cam = (cam - cam.min()) / (cam.max() + 1e-5)
    cam_resized = cv2.resize(cam, (W, H))
```

```
    heatmap = plt.cm.jet(cam_resized)[:,:,:3]
    overlay = np.clip(0.4*heatmap + 0.6*img_np, 0, 1)
    ax.imshow(overlay)
    ax.set_title(f"ch␣{ch}")
    ax.axis("off")

plt.suptitle("VGG16␣last␣conv␣-␣per-channel␣Grad-CAM␣(top␣15)")
plt.tight_layout()
plt.show()
h1.remove(); h2.remove()
```

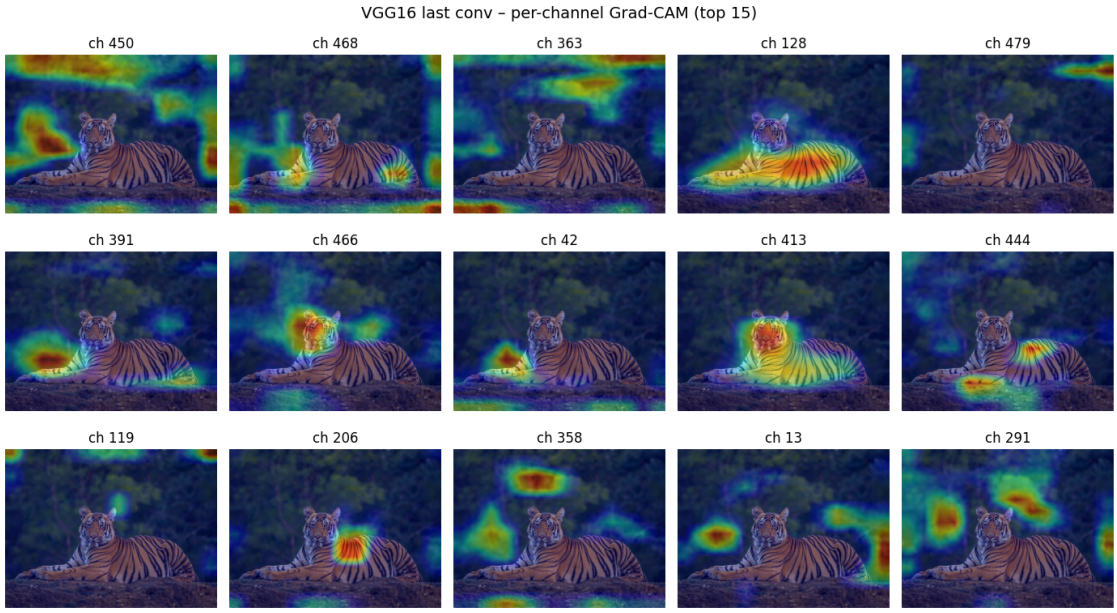


Figure 3: VGG16 last convolutional layer – top 15 per-channel Grad-CAM overlays for the tiger image. Each channel captures a distinct semantic region (e.g., stripes, torso, face).

Your tasks

(a) **ResNet50 activations (2.5 pts).**

Using a pretrained **ResNet50** and the **same two input images** (tiger and face), extract and visualize feature maps at three depths:

- **Early layer:** e.g. output of `layer1`
- **Mid layer:** e.g. output of `layer2` or `layer3`
- **Deep layer:** e.g. output of `layer4` (use Grad-CAM if the maps are not interpretable)

For each depth, display at least **16 channels** as a 4×4 grid (like in Fig. 2). Do this for **both** the tiger and the face image. Clearly label each figure with: model name, image name, and layer name.

(b) **DenseNet121 activations (2.5 pts).**

Repeat the same procedure using a pretrained **DenseNet121**. Extract and visualize feature maps from:

- **Early block:** `features.denseblock1`
- **Mid block:** `features.denseblock2` or `features.denseblock3`
- **Last block:** `features.denseblock4` (use Grad-CAM for interpretation)

Again, show at least 16 channels (4×4) for both images.

(c) **Analysis and comparison (5 pts).**

i. **Abstraction (1 pt):** Describe how the feature maps evolve from **early** $\rightarrow$ **mid** $\rightarrow$ **deep** layers in both ResNet50 and DenseNet121.

ii. **Sparsity (1 pt):** A sparse activation map has only a few bright, active regions (most values near zero), indicating that the network focuses on specific discriminative areas rather than the entire image. Discuss whether the tiger or face image yields sparser activations, and relate it to each model's architecture and feature reuse.

iii. **Smoothness / resolution (1 pt):** Deeper feature maps often appear "blocky" or coarse. This occurs because spatial dimensions progressively shrink through pooling or strided convolutions (e.g., $224 \times 224 \rightarrow 14 \times 14$), meaning each activation corresponds to a large image region (a large receptive field). Explain how this reduced spatial resolution limits fine-grained detail but helps capture global semantic context.

iv. **Semantic focus (1 pt):** Using Grad-CAM, compare which semantic regions the networks emphasize in their final layers (e.g., tiger's face vs body, human facial center vs hair or background). Note whether ResNet50 and DenseNet121 focus on similar or different regions, and hypothesize why.

v. **Image dependence (1 pt):** Compare interpretability between the two input images. Which input (tiger vs face) produces more structured or meaningful deep activations? Discuss why one image might yield clearer activations.

Question 2: Multi-Car Speed Estimation with Detection & Optical Flow (10 pts)

Question 2 Resources

In this question you will work with a short **highway video** (approximately 60 seconds long, **1501 frames at 25 FPS**) where multiple cars move through a fixed camera view. Your goal is to (1) detect cars, (2) track them across frames, and (3) estimate their relative motion speed using dense optical flow.

Your tasks

(a) **Car detection (3 pts).**

Run an object detector on every frame of the video and keep only vehicle classes (`car`, `truck`, `bus`). Draw bounding boxes and class labels on each detected vehicle.

(b) **Tracking (3 pts).**

Maintain a consistent ID for each vehicle across frames (i.e., the same car should have the same ID as it moves through the video). Draw tracking lines for each vehicle and track it throughout the video.

(c) **Speed estimation (4 pts).**

For every frame, estimate the motion of each tracked vehicle and express it as a **mean displacement in pixels per frame**. Convert this value to **pixels per second (px/s)** using the known frame rate of the video:

$$v_{\text{px/s}} = (\text{displacement in px/frame}) \times 25$$

For example, a car with 8.46 px/frame corresponds to $8.46 \times 25 = 211.5$ px/s.

Display the computed **px/s** value above or next to each vehicle in the video frames.

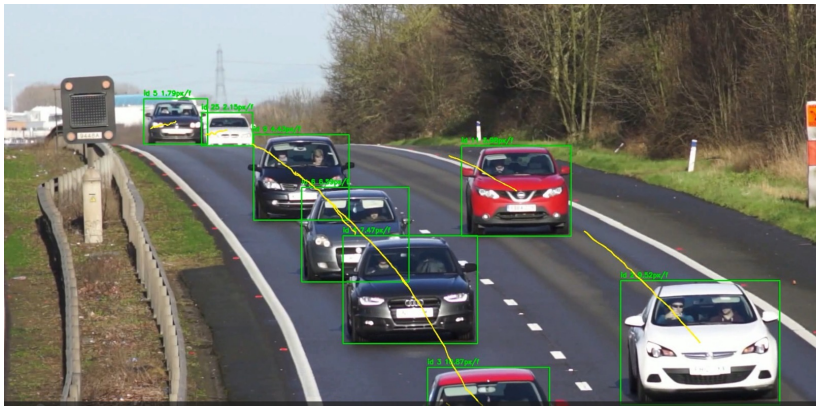


Figure 4: Example multi-car tracking and speed overlay. Each label shows a tracker ID and the corresponding motion in pixels per second (px/s).

Question 3: Person & Dog Tracking with Object Detection & Motion Analysis (15 pts)

Question 3 Resources

In this question, you will analyze a short video of a **Dog running towards a Person**. Your goal is to detect and track both the person and the dog across all frames, visualize their trajectories, and estimate their motion magnitudes over time.

Your tasks

(a) **Detection (3 pts).**

Run an object detector on each frame and retain only the **person** and **dog** classes. Draw bounding boxes and class labels in different colors (for example, red for the person and yellow for the dog). Save 3–4 representative annotated frames.

(b) **Tracking (4 pts).**

Maintain consistent IDs for both the person and the dog throughout the video. Show that each keeps the same ID across frames even when they partially overlap or move apart.

(c) **Motion estimation (4 pts).**

For each frame, estimate the motion of both tracked objects and express it as **mean displacement in pixels per frame**.

(d) **Trajectory visualization (2 pts).**

For the entire video, record the center of the bounding box of the dog and plot it's trajectory as a continuous line over time. Ensure the dog's path is visible from the first frame to the last frame.

(e) **Analysis (2 pts).**

Ensure that the dog remains correctly tracked even when it becomes partially or fully occluded, demonstrating robust ID consistency throughout the sequence.



(a) Person and dog tracking (early frame).



(b) Person and dog tracking (later frame).

Figure 5: Example of person and dog tracking with trajectory lines. Each label shows tracker ID and motion in pixels per second (px/s).

Question 4: Penalty Kick Prediction Kaggle Competition (65 pts)

Predict the Penalty... Before the Foot Meets the Ball

Kaggle Invitation Link

Can a model anticipate a penalty kick before the foot reaches the ball? A penalty taker has a fraction of a second to commit to a side, and the goalkeeper has to guess at the same time. In this competition, your goal is to build a model that uses a short video ending **before annotated foot–ball contact** to predict the shot direction, the goalkeeper's movement, and whether the kick results in a goal.

You are given **1,300 labeled public clips** to train and validate on, and you submit predictions for **900 unlabeled private clips**. All clips, public and private, contain **only pre-contact footage**, so models are trained and evaluated without ever seeing the kick or its aftermath. The full competition README, packaged with the data, repeats everything below with more detail.



Figure 6: Sample penalty

What Are You Predicting?

For every private clip, you must predict three labels:

1. **Kick direction** (`kick_direction`): `left`, `middle`, or `right`.
2. **Goalkeeper direction** (`goalkeeper_direction`): `left`, `stands in the middle`, or `right`.
3. **Goal outcome** (`goal_outcome`): `goal` or `no_goal`.

PLEASE NOTE: left and right are defined **from the kicker's perspective** while facing the goal, for both the kick and the goalkeeper. A keeper who dives to the same side as the ball therefore has the same label as the kick. The binary outcome maps the source annotations `save`, `out`, and `post` to `no_goal`.

Evaluation

Submissions in this competition are evaluated using a weighted multi-target accuracy over the N scored private clips, combining:

1. Kick Direction (60%)

$$\text{Accuracy}_{\text{Kick}} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{k}_i = k_i)$$

2. Goalkeeper Direction (25%)

$$\text{Accuracy}_{\text{GK}} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{g}_i = g_i)$$

3. Goal Outcome (15%)

$$\text{Accuracy}_{\text{Goal}} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(\hat{o}_i = o_i)$$

where:

- $\hat{k}_i, \hat{g}_i, \hat{o}_i$ are the predicted labels and k_i, g_i, o_i the true labels
- $\mathbb{I}(\cdot)$ is the indicator function (1 if the condition is true, 0 otherwise)
- N is the number of scored clips

Final Metric

The final competition score is a weighted combination of the three accuracies:

$$\text{Score} = 0.60 \cdot \text{Accuracy}_{\text{Kick}} + 0.25 \cdot \text{Accuracy}_{\text{GK}} + 0.15 \cdot \text{Accuracy}_{\text{Goal}}$$

The score ranges from 0 to 1; higher is better.

$0.60 \cdot 0.467 + 0.25 \cdot 0.515 + 0.15 \cdot 0.722 = 0.517$ on the public data. The bar you must actually beat is higher; see **Grading** below.

Submission Format

Your submission must be a CSV with one row per private clip:

```
clip,kick_direction,goalkeeper_direction,goal_outcome
001_private.mp4,left,right,goal
002_private.mp4,right,stands in the middle,no_goal
...
```

Please refer to the provided `submission_template.csv`.

Constraints:

- `clip` must match `private_metadata.csv` exactly, including the `.mp4` extension.
- Each prediction must use one of the exact class strings listed above.
- Every private clip must appear exactly once (900 rows plus a header).

Leaderboard Split

Both Kaggle leaderboards are computed on the 900 private clips only; the public clips are never scored. During the competition, the **public leaderboard** shows your score on a fixed random **40%** of the private clips (360 clips); which clips are in that 40% is not disclosed. The **final ranking** uses the remaining **60%** (540 clips, the **private leaderboard**) and is revealed when the competition closes. Because the public leaderboard is only part of the data, rely on your own source-grouped validation rather than tuning to it.

- **Team size:** at most 2 students.
- **Submission limit:** at most 20 submissions per team per day.
- **External data and pretrained models:** any pretrained model and any external dataset may be used; name them in your notebook.
- **Number of final submissions:** you may select a single final submission

Grading (65 pts)

Component	Points	How it is earned
Competition	50	Based on your final private-leaderboard score
Report	10	Your notebook's explanation of methods, results, and challenges
Leaderboard SOTA	5	Held first place on the leaderboard at any time during the competition
Total	**65**	

IMPORTANT: Beat chance or score ZERO on the competition.

1. **No signal, no points.** If your final private-leaderboard score does **not beat chance**, you receive **0 of the 50 competition points**, however much work went into the model.
2. **Chance is an educated guess, not random guessing.** The educated guess scores **0.52277** on the public data. The bar you must beat is the educated guess on the **final private leaderboard**. Your model has to extract real signal from the video.
3. **SOTA also requires beating chance.** You must beat the chance score to be eligible for any SOTA credit.

Dataset Description

The 2,200 clips come from 215 source videos (broadcast shootouts, match highlights, and penalty compilations) from men's and women's competitions.

Partition	Clips	Files and labels
Public	1,300	Pre-contact clips; all target labels
Private	900	Pre-contact clips only; metadata, no labels

Pre-contact clips contain every frame whose timestamp is at or before the annotated contact time minus **0.02 seconds**, and nothing after it. They start at the beginning of the original clip, keep its resolution and frame rate, and have no audio. Their length ranges from about 0.3 s to 9.8 s (median about 1.9 s). Public and private clips are cut the same way; no full clips showing the kick or its aftermath are provided.

Source IDs. Each clip has an anonymous `source_id`; clips sharing a `source_id` were cut from the same source video (the same shootout or the same compilation). Almost every source lies entirely in one partition (133 public-only and 79 private-only sources), so most private clips come from sources you have never seen. Three long multi-match compilations contribute clips to both partitions, mostly public. Validate with groups by `source_id` (e.g. `GroupKFold`); a random clip-level split leaks the same shootout into train and validation and overstates accuracy.

Note: Most sources come from penalty shootouts some are in game penalty footage, being mindful of this distinction may be helpful.

Dataset Structure

- `public/<clip>`: labeled public clips, `0001_public.mp4` ... `1300_public.mp4`
- `private/<clip>`: unlabeled private clips, `001_private.mp4` ... `900_private.mp4`, same format as `public/`

Clip numbers are assigned at random: neighbouring numbers are not related and do not reveal the league, match, or date.

CSV Files

`public_ground_truth.csv` contains the metadata and true labels for all 1,300 public clips.

Column	Description
`clip`	Clip file name, e.g. `0001_public.mp4`
`source_id`	Anonymous source video ID, e.g. `src_067`
`preferred_foot`	Recorded preferred foot of the kicker (`left`/`right`)
`gender`	`M` (men's) or `F` (women's) soccer
`resolution`	Width × height of the clip (px)
`fps`	Average frame rate of the clip
`kick_direction`	`left`, `middle`, or `right`
`goalkeeper_direction`	`left`, `stands in the middle`, or `right`
`goal_outcome`	`goal` or `no_goal`
`outcome_detail`	`goal`, `save`, `out`, or `post` (not scored)

`private_metadata.csv` contains the same columns for the 900 private clips **except** the label fields: `kick_direction`, `goalkeeper_direction`, `goal_outcome`, `outcome_detail`.

Note: `preferred_foot` is guaranteed to be the foot used for a particular kick.

`public_position_clicks.csv` (supplemental) gives hand-annotated kicker and goalkeeper positions for 879 public clips: `clip`, `kicker_click_time_seconds`, `kicker_x_norm`, `kicker_y_norm`, `keeper_click_time_seconds`, `keeper_x_norm`, `keeper_y_norm`. Positions are fractions of the frame width and height from the top-left corner, clicked on the frame 0.25 s before foot–ball contact. No clicks are provided for private clips, so use them for training or analysis, not as a model input at prediction time.

`submission_template.csv` lists the 900 private clips with empty prediction columns: `clip`, `kick_direction`, `goalkeeper_direction`, `goal_outcome`.

Label Distribution (Public)

kick_direction		goalkeeper_direction		goal_outcome	
left	607	left	669	goal	939
middle	181	stands in the middle	63	no_goal	361
right	512	right	568		

Rules

- Predictions for a private clip may use only its pre-contact video and its row in `private_metadata.csv`, plus what your model learned from the public data, external data, and pretrained models.
- Any pretrained model and any external dataset may be used, as long as you name them in your notebook.
- No manual labeling of private clips and no human in the loop at prediction time.
- The one limit on external data: do not search for the original broadcasts or other footage or records of these penalties to look up their outcomes.
- Your notebook must reproduce your final submission: training, inference, and the CSV it writes.

Prediction Targets

Models must predict:

- `kick_direction`: `left`, `middle`, or `right`
- `goalkeeper_direction`: `left`, `stands in the middle`, or `right`
- `goal_outcome`: `goal` or `no_goal`

All three predictions are required for every private clip.